Survive the Semester

Project Report

CT120-4-1-DSA

Group Members:

No.	Name	TP Number	Intake Code
1.	MIN NYO SIN	TP138020	APD1F2511CGD(GT)
2.	AHMED UWAIS IBRAHIM	TP076328	APU1F2511CGD(GT)

Lecturer : **Jacob Sow Tian You**

Module : **Data Structures and Algorithms**

Date Assigned : **May 28, 2026**

Date Completed : **August 20, 2026**

Table of Contents

Table of Figures

Table of Tables

1. Introduction

(Contributed by: MIN NYO SIN)

1.1. Project Concept and Motivation

Survive the Semester is a text-based RPG written in C++ for this Data Structures and Algorithms project. The idea started from a simple question: instead of fighting monsters, could the player fight the problems that normally appear during a university semester? This gave us a setting that was different from a normal RPG, but still suitable for demonstrating data storage, searching, traversal, and repeated game loops.

We kept the familiar turn-based idea. The player chooses an action first, and the current challenge responds afterwards. The opponents are situations such as Phone Distraction, a Difficult Lecture, a Surprise Quiz, Group Assignment problems, Procrastination, and the Final Examination. We did not want every encounter to feel like the same battle with a different name, so their choices, costs, and responses were changed to match the situation.

Four statistics are used throughout the game: Focus, Knowledge, Energy, and Motivation. They are not only values shown on the screen. They limit what the player can do. For example, studying can improve Knowledge but consumes Energy, while taking a break gives the player time to recover but does not reduce the challenge level. This makes each turn a small decision instead of simply choosing the strongest option every time.

After building the first encounter, we felt that moving directly from one battle to another would make the game too narrow. For this reason, exploration was added. The player can move between APU and KLCC, attend class, explore, visit a shop, find part-time work, and use items. Money earned earlier can be spent on Coffee, Energy Drinks, Study Notes, and other items that may help in a later encounter.

The data structures were connected to these gameplay features rather than being added as separate demonstrations. A custom singly linked list stores the inventory, while the small shop catalogues use fixed arrays. The program also makes use of linear search, list traversal, random selection, validation loops, and checkpoint copying. These parts work together whenever the player buys an item, uses it, or retries a failed week.

The final result is a small but complete student-life RPG. It shows that topics normally introduced through simple number examples can also support an interactive program with a story, changing data and consequences between stages.

1.2. Scope and Theme

The theme is based on situations that university students can easily recognize. Losing focus while studying, finding a lecture difficult, depending on group members, and preparing too late for an examination are all used as game events. Because the situations are familiar, the player can apply some real-life reasoning when choosing an action.

Since the project is text-based, most of the atmosphere has to come from dialogue, short scene descriptions, statistics, and ASCII artwork. The player enters a name and makes the decisions directly, which helps the story feel personal even without a graphical world. The result of a choice can be seen immediately through changes to Focus, Energy, Knowledge, Motivation, money, or inventory.

The playable timeline begins in Week 1 and ends with the Final Examination in Week 15. It contains several encounters, exploration periods, two main locations, shops, and an inventory. Checkpoints are placed before important stages. Therefore, losing all Focus does not always mean replaying the entire game; the player can restart the current period instead.

Our first plan included more weeks, locations, and random events. It was too large for the available development time, so only the more important weeks were developed in detail, and the remaining weeks are connected through short transitions. Reducing the scope gave us enough time to finish and test the inventory, encounters, exploration, and retry system instead of leaving many incomplete features.

2. Design and Techniques

(Contributed by: MIN NYO SIN)

2.1. Overall System Design

The project is separated into several header and source files. We chose this structure because the main program became difficult to follow once encounters, exploration, and shops were added. `main.cpp` now concentrates on moving the player through the semester, while the gameplay details remain in their own modules.

`Player` is the shared object between these modules. It holds the statistics, money, inventory, and a few progression values that must remain available from one week to another. Encounter, exploration, and shop functions receive the same `Player` pointer. As a result, an item bought at KLCC or Knowledge earned in class is still present when the next challenge begins.

The main components of the system are as follows:

- `main.cpp` moves between weeks, reads the returned results, and controls retry decisions.
- `Player.h` keeps the player's statistics, money, inventory, and related player functions.
- `Challenge.h` contains the common challenge values: level, pressure, and remaining time.
- `Encounters.cpp` contains the turn-based loops and the different responses used by each challenge.
- `Exploration.cpp` manages location menus, one-time activities, and movement between APU and KLCC.
- `Inventory.h` and `Item.h` store the items owned by the player and their statistic effects.
- `Shop.cpp` displays the products, checks the player's money, and sends a purchased `Item` to the inventory.
- `Validations.cpp` clears failed input so that a wrong value does not break the menu loop.
- `Display.cpp` and `Ascii.cpp` handle slow text, headings, artwork, and other console output.

The usual data flow starts in `main.cpp`. It calls an encounter or exploration function and passes the `Player` pointer. That function reads a choice, changes the `Player` data, and

returns a result. In our encounter functions, **1** means the challenge was completed, **2** means time ended while the player was still alive, and **3** means the player's focus reached zero. `main.cpp` uses this result to choose the next part of the program.

Checkpoints were added because restarting from Week 1 after every loss would make the later part of the game frustrating. Before an important period, `copyPlayerData()` creates a separate copy of the current `Player` data. If a retry is selected, that copy is written back to the active player. The inventory requires a deep copy, which is discussed in the data structure section.

This organization also made the project easier to extend. For example, a new encounter can be added to `Encounters.cpp` and called from `main.cpp` without rewriting the inventory or shop.

2.2. Overall Gameplay Flow

The gameplay follows the progress of a student from the beginning until the end of a semester. At the start of the game, the player enters a name, and a Player object is created with the starting statistics, money, and inventory. The player then progresses through several important periods of the semester.

The main gameplay stages are as follows:

1. **Week 1 – Phone Distraction**

   The first encounter takes place while the player is trying to study after returning from class. Phone Distraction works as an introduction to the round system: choose an action, pay its cost, and then deal with the phone's response.

2. **Week 2 – APU and KLCC Exploration**

   Week 2 opens the game for the first time. The player may start at APU or KLCC and can move between them until going home. APU includes the class encounter and campus shop, while KLCC includes its own shop, exploration events, and part-time work. Going to KLCC before class means the class is missed.

3. **Weeks 3 to 5 – Semester Progression**

   Weeks 3 to 5 are shown through a short story transition rather than three repeated menus. This keeps the pace moving towards the next important event in Week 6.

4. **Week 6 – Surprise Quiz and Exploration**

   The player faces a more difficult Surprise Quiz encounter. Once it is over, the player receives another exploration period and can prepare before the longer Group Assignment section starts.

5. **Weeks 7 to 12 – Group Assignment**

   The Group Assignment is one challenge spread across six turns, with each turn representing another week. This allowed the messages and group-member problems to change from Week 7 until Week 12 instead of presenting the assignment as one short fight.

6. **Week 13 – Final Preparation**

   Week 13 is the last open preparation period. The player can improve statistics, work for money, buy items or return home when ready. The state at the start of this week is also important because the Final Examination menu can send the player back here.

7. **Week 14 – Procrastination**

   During Week 14, the player stays home to revise and faces Procrastination. Completing it provides more preparation for the examination. Running out of challenge time does not immediately end the semester, but the player reaches Week 15 with less Knowledge than they could have earned.

8. **Week 15 – Final Examination**

   The Final Examination is the last major challenge. It mainly depends on the knowledge accumulated by the player throughout the semester. Passing the examination completes the game successfully.

Most encounters return a result that tells the main program whether the player won, failed while still surviving, or lost all focus. When the player loses all focus, the checkpoint and retry system allows the current stage to be restarted. If the Final Examination is failed, the

player is given the choice to quit, retry the examination, or return to Week 13 and prepare again. Below is the flowchart of the game. A clear version (link) is provided in the Appendix.

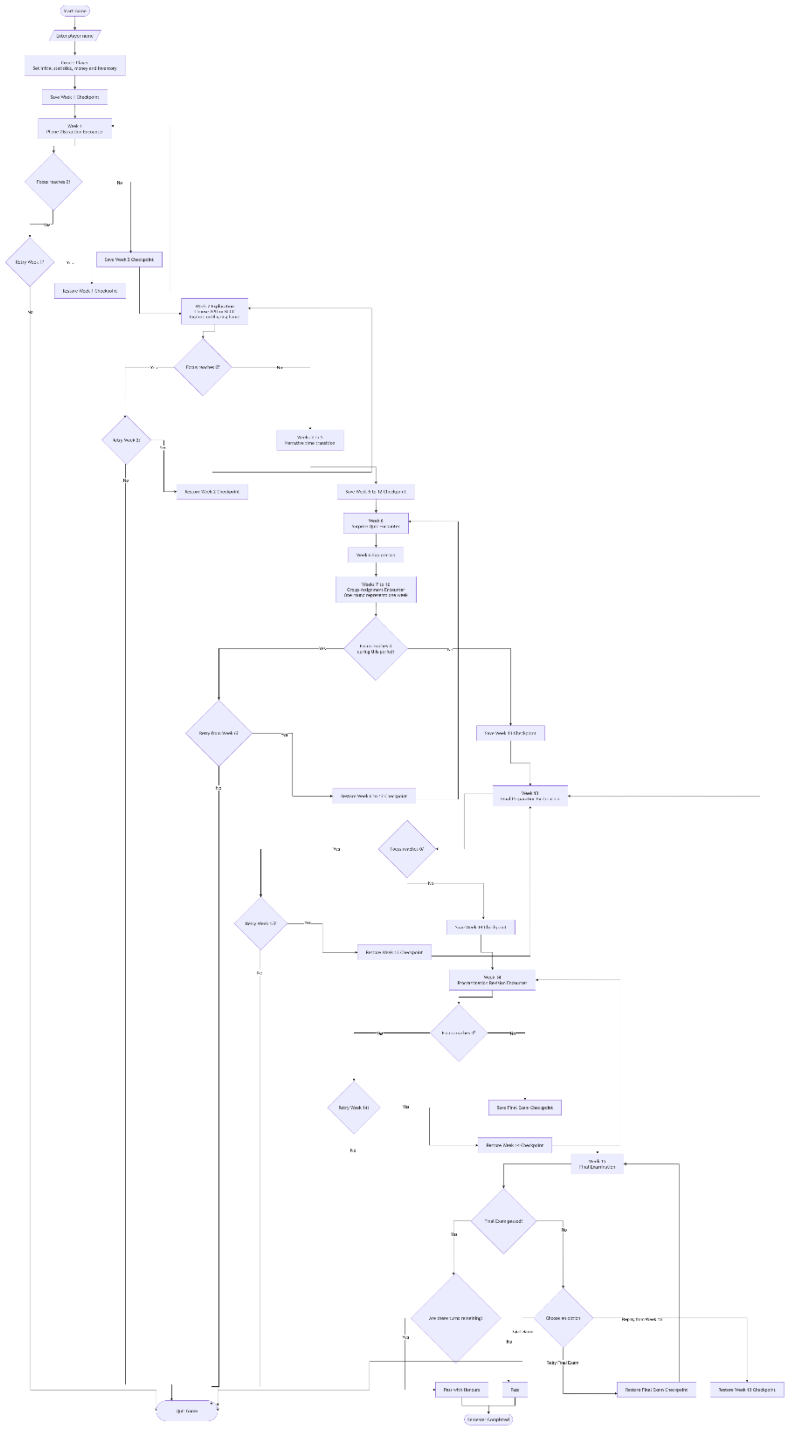

Figure 4: Program Flowchart

Although the game follows a main semester timeline, the player is still given different choices during encounters and exploration. These choices affect statistics, available items, money, and preparation for later challenges. Therefore, the gameplay flow remains structured while still allowing different results depending on the decisions made by the player.

2.3. Data Structure Used

The data structures in **Survive the Semester** were selected according to the kind of information being stored. We used struct for values that belong together, fixed arrays for shop catalogues, and a singly linked list for the inventory. The checkpoint system also required a separate copy of the Player data because the inventory contains dynamically allocated nodes.

2.3.1. Item and Statistic Structures

The `Item` structure was created when the shop and inventory systems were added. Before this, item properties would have needed to be stored in separate variables, which would make passing an item between the shop, inventory, and Player functions inconvenient.

Each `Item` stores its name, price, and the amount of Focus, Knowledge, Energy, and Motivation it restores. The values may also be negative. For example, an Energy Drink restores a large amount of Energy but reduces Focus. Using one structure means that the same purchasing and item-usage functions can handle every product.

`PlayerStats` and `ChallengeStats` serve a different purpose. They temporarily record statistics before an action occurs. The saved values are later compared with the updated statistics so that the game can display the result of one round or activity.

```
Energy: 80 -> 70 (-10)

Knowledge: 20 -> 22 (+2)
```

We used structures instead of classes for these values because they mainly store related data. They do not require a large set of independent behaviors.

2.3.2. Fixed Arrays for Shop Items

Each shop uses a fixed array containing three products. A fixed array was suitable because the shop catalogue is decided in the source code and does not change while the game is running.

The menu selection can be connected directly to the array using:

```
items[option - 1]
```

The subtraction is necessary because the menu begins from one while the array begins from index zero. This lets the same `buyShop()` function process any selected item.

A dynamic structure was not necessary for the shop because products are not inserted or removed during gameplay. Even when Study Notes become sold out, the product remains part of the catalogue; only its available quantity changes.

2.3.3. Singly Linked-List Inventory

The inventory is implemented using a singly linked list. Each node stores an `Item`, its owned quantity, and a pointer to the next node. The list is managed using `head`, `tail`, and `size`.

We selected a linked list because the number of item types owned by the player changes during the game. A new node can be created when a new item type is purchased, and a node can be deleted when its quantity reaches zero. The inventory does not require a fixed capacity, and removing a node does not require shifting the remaining items. For this small game, a fixed array could also store the six available product types. However, the linked list gave us a useful place to apply dynamic memory, pointer traversal, insertion, and deletion. Its slower indexed access is acceptable because the inventory is small.

Only one node is used for each item type. If the player buys another Coffee, the quantity in the existing Coffee node is increased instead of creating a duplicate node. This keeps the displayed inventory shorter and makes item quantities easier to understand. The `tail` pointer was included so that a new item type can be inserted directly at the end. Without it, the program would need to traverse the complete list before every end insertion.

The inventory is stored directly inside the Player class. This is composition rather than inheritance: a Player owns an inventory, but an inventory is not a type of Player. Nodes are

created using `new` and removed using `delete`. The inventory destructor calls `clear()` to delete any remaining nodes when the Player is destroyed.

2.3.4. Deep-copy Checkpoint Structure

The checkpoint system stores the player's condition before an important stage. This includes statistics, money, inventory quantities, and progression values such as the part-time job status and remaining Study Notes.

A normal shallow copy was not suitable because the inventory contains pointers. Copying only the pointer addresses would make the active Player and checkpoint refer to the same nodes. Using an item during the current attempt could then also change the checkpoint, which would make retrying unreliable. Deleting both Players could also attempt to delete the same memory.

The `copyPlayerData()` function therefore performs a deep copy. Normal Player values are copied directly, while the source inventory is traversed and each item is inserted into a newly created destination node.

The two inventories may contain identical information, but their nodes are stored separately:

```
Active Player: [Coffee x2] -> [Study Notes x1] -> nullptr

Checkpoint: [Coffee x2] -> [Study Notes x1] -> nullptr
```

When a retry is selected, the checkpoint is copied back into the active Player. Items used, money spent, and statistics lost during the failed attempt are returned to their saved values.

This approach requires more memory than a shallow copy, but the extra memory is necessary because the checkpoint must remain unchanged while the player continues through the stage.

2.4. Algorithms used

The algorithms in the project control inventory operations, encounters, exploration, and retries. We selected relatively simple algorithms because the amount of game data is small, but each algorithm still plays a specific role in maintaining the correct game state. Detailed complexity analysis is provided in Section 2.5, while the related code is discussed in Section 3.

2.4.1. Linear Search

`addItem()` uses linear search to determine whether a purchased item already exists in the inventory. The search begins from the `head` and compares each node's item name with the purchased item. If a match is found, the node's quantity is increased. If the search reaches `nullptr`, the item is new, and another node is inserted.

We selected linear search because a singly linked list must be followed one node at a time. The game also has only a small number of item types, so using a more complicated search structure would provide little practical advantage. The main purpose of the search is to prevent duplicate nodes such as several separate Coffee entries.

2.4.2. Linked-List Traversal

Linked-list traversal is used whenever several inventory nodes must be processed. A temporary pointer begins at `head` and repeatedly moves to `next`.

This traversal appears in:

- `displayInventory()`, to show all owned items
- `getItemAt()`, to reach the selected position
- `clear()`, to delete every node
- `copyPlayerData()`, to copy the inventory into a checkpoint

Linear search and traversal use similar pointer movement, but their goals are different. Search may stop after finding a matching item, while a full traversal continues until all required nodes are processed.

Traversal is necessary because linked-list nodes are not stored next to each other like array elements. There is no direct equivalent to items[index]; the program must follow the links from the beginning.

2.4.3. Insertion and Removal

A new item type is inserted using `insertAtEnd()`. The function creates a node, stores the item and quantity, and sets its `next` pointer to `nullptr`. If the inventory is empty, both `head` and `tail` point to the new node. Otherwise, the node is connected after the current tail, and `tail` is updated. Keeping the tail pointer makes end insertion direct and avoids another complete traversal.

Removal happens when the player uses the last quantity of an item. Removing the first node changes `head`, while removing another node requires the program to locate the previous node and reconnect its `next` pointer.

The final-node and only-node cases require additional attention because `tail` may also need to change. These separate conditions prevent the inventory from keeping pointers to deleted nodes.

2.4.4. Encounter-loop Algorithm

Every major challenge uses a turn-based encounter loop. The loop continues while the challenge has level and time remaining and the player still has Focus.

During one round, the program:

1. Saves the current player and challenge statistics.
2. Displays the available actions.
3. Validates and processes the player's choice.
4. Updates the player or challenge.
5. Checks whether the challenge has been defeated.
6. Allows a challenge response when required.
7. Reduces the remaining time.
8. Displays the round changes and checks the ending conditions.

The temporary statistics are stored inside the loop because they must represent the beginning of the current round rather than the beginning of the complete encounter. Some

actions calculate progress using Energy, Motivation, Focus, and Knowledge. This makes the player's preparation important. Other actions have a fixed effect but may consume a particular resource.

The `isYourTurn` Boolean controls whether the challenge responds. For example, some defensive or communication actions prevent a response, while a normal study action may allow another distraction afterwards.

We check whether the challenge has been defeated before processing its response. Otherwise, a challenge with zero level could still damage the player after being defeated.

Each encounter returns one common result:

- `1` for challenge completed
- `2` for timeout while the player survives
- `3` for the player losing all Focus

These result codes allow `main.cpp` to handle every encounter in the same way without reading its internal variables.

2.4.5. Exploration State Algorithm

Exploration uses a loop controlled by the current location and several Boolean flags. During Week 2, for example, `currentLocation` records whether the player is at APU or KLCC. Changing the location does not end exploration. The loop repeats and displays the menu belonging to the new area. Exploration finishes only when the player chooses to return home or loses all Focus.

Boolean flags record activities such as attending class, missing class, exploring an area, or finding a part-time job. These flags prevent the same one-time reward from being collected repeatedly. The missed-class flag was particularly important. If the player visits KLCC before attending class, returning to APU does not reopen the morning class. The player may still use the other campus activities.

Energy is checked before study, class, work, and exploration actions. When Energy reaches zero, those activities are blocked, but the inventory, shops, and Go Home option remain available. This prevents the player from becoming stuck without a recovery choice.

We used simple integers and Boolean values instead of creating another class for exploration states. There are only two main locations and a small number of activities, so the simpler approach was easier to follow.

2.4.6. Random-selection Algorithm

Random selection is used for enemy responses and exploration events. The program uses `random_device`, `mt19937`, and `uniform_int_distribution` to generate a number within a fixed range.

The number is connected to an event through `if` and `else if` conditions. This means that the same player choice may lead to a different distraction, reward, or penalty during another playthrough. Some functions give one number to each event, creating equal chances. In other cases, several generated values reach the final `else`, which makes that result more common. This provides a simple form of weighting without adding a separate probability system.

Random events were included to reduce repetition. The player can learn the possible results, but cannot know exactly which one will occur during a particular round.

2.4.7. Checkpoint and Retry Algorithm

A checkpoint is created before an important week or period begins. The current Player values and inventory are copied into a separate Player object.

A Boolean flag controls whether the stage has finished. If an encounter returns `1` or `2`, the flag becomes true, and the game continues. If the result is `3`, the player is asked whether to retry. Choosing retry copies the checkpoint back into the active Player and repeats the same loop. Choosing the “no” option ends the game and deletes the related objects. We used `while` loops and flags rather than restarting the complete program. This made it possible to retry only the current stage. It also became easier to support more than one retry point.

The Final Examination uses two checkpoints. One restarts only the examination, while the Week 13 checkpoint repeats the final preparation period. This gives the player a choice between trying the boss again immediately and returning to improve their statistics.

2.4.8. Validation and Edge-case Algorithms

Input validation handles two different problems. First, `isInputValid()` checks whether `cin` failed because the player entered text instead of a number. It clears the stream and returns control to the current menu.

Second, range checks handle numbers that can be read successfully but do not belong to the menu. These values are rejected using conditions or the `default` case of a `switch`.

Gameplay boundaries are checked separately. These include:

- Insufficient money or Energy
- Invalid inventory indexes
- Empty inventory usage
- Study Notes being sold out
- Item quantity reaching zero
- Repeated exploration activities
- Statistics moving outside their limits
- A defeated challenge attempting another response

The `checkStats()` function keeps player statistics within their valid ranges after encounters, items, and exploration events. Pointer checks in the inventory handle the first, final, and only-node deletion cases.

We kept these checks close to the system they protect. General input failure is handled by `Validations.cpp`, player-stat limits are handled by Player, and linked-list boundaries are handled by the inventory. This separation made the errors easier to locate during testing.

2.5. Time and Space Complexity Analysis

Time complexity describes how the running time of an algorithm changes when the input size increases, while space complexity describes how the amount of memory required changes with the input size (Cormen et al., 2022; Weiss, 2014). Big O notation is used because it focuses on the growth of an algorithm instead of the exact execution time, which may be affected by the computer, compiler, and console output speed (Cormen et al., 2022; Weiss, 2014).

For this analysis, different alphabets will be used for different cases. Also, n will represent the number of different item nodes in the inventory. The current game only contains six purchasable item types. Therefore, the actual inventory will always remain small. However, n is treated as a variable to show how the inventory would perform if more items were added later.

The `slowPrint()` delay and the time spent waiting for user input are not included. Complexity analysis focuses on the operations performed by the program.

2.5.1. Inventory Search and Traversal

The `addItem()` function uses linear search to check whether an item already exists. It starts from the `head` and compares the purchased item with each node.

In the best case, the item is found in the first node. This takes constant time $O(1)$.

In the worst case, the item is in the final node or does not exist. The function must check all `n` nodes. Therefore, the worst-case time complexity of `addItem()` is $O(n)$.

The function only uses one temporary pointer named `current`. The number of temporary variables does not increase with the inventory size, so its auxiliary space complexity is $O(1)$.

The same traversal method is used by several other functions.

The `displayInventory()` function always visits every node to display all items. Its time complexity is $O(n)$.

The `getItemAt()` function moves from the `head` until it reaches the selected index. Accessing the first node takes $O(1)$, while accessing the final node takes $O(n)$ in the worst case.

The `clear()` function visits and deletes every node. Its time complexity is also $O(n)$. Since it deletes the nodes one at a time and only uses one temporary pointer, its auxiliary space remains $O(1)$.

2.5.2. Insertion and Removal

The `insertAtEnd()` function inserts a new item after `tail`. Since the inventory already stores a tail pointer, the program does not need to search for the last node.

The function performs a fixed number of operations:

1. Create a new node.
2. Store the item and quantity.
3. Connect the new node to the list.
4. Update `tail`.
5. Increase `size`.

These steps do not depend on the number of existing nodes. Therefore, insertion at the end takes $O(1)$ time

Each insertion creates one new node, so it adds $O(1)$ space. If the inventory contains n different item nodes, the complete linked list requires $O(n)$ space

The `removeItemAt()` function has different results depending on the selected position. Removing the first node takes $O(1)$ because `head` can be changed directly.

Removing a middle or final node requires the program to locate the previous node. In the worst case, it must traverse almost the complete list. Therefore, the worst-case removal time is $O(n)$

The removal algorithm only uses pointers such as `previous` and `toDelete`. Its auxiliary space complexity is $O(1)$.

2.5.3. Item Usage and Inventory Menu

The `useItem()` function first calls `getItemAt()` to find the selected item. This may take $O(n)$ time.

If the quantity becomes zero, it also calls `removeItemAt()`, which may perform another traversal. The complete worst-case time is:

$$O(n) + O(n) = O(n)$$

The two traversals do not make the result $O(n^2)$. They happen one after another, so the result is $O(2n)$, which simplifies to $O(n)$.

The `openInventory()` function displays the inventory and may then use an item. Displaying the list takes $O(n)$, and using an item also takes up to $O(n)$. Therefore, the overall worst-case time complexity of opening and using the inventory remains $O(n)$.

If the inventory is empty, the function returns immediately, giving a best-case time of $O(1)$.

2.5.4. Shop and Fixed-Array Complexity

Each shop stores three products inside a fixed array:

```
Item items[3];
```

The selected product is accessed using:

```
items[option - 1]
```

Array indexing accesses the required element directly. It does not check the earlier items first. Therefore, selecting a product from the shop array takes $O(1)$ time.

Since the array always stores three elements, its space requirement is also $O(1)$.

The complete purchase may take longer because `buyShop()` calls `addItem()`. Checking the player's money takes $O(1)$, but searching the inventory takes up to $O(n)$. Therefore, purchasing an item has a worst-case time complexity of $O(n)$.

If the player makes several purchases or repeatedly opens the inventory while inside the shop, the total time also depends on the number of shop choices.

2.5.5. Encounter-Loop Complexity

The encounter loop continues while the challenge has level and time remaining and the player still has Focus.

A normal encounter action performs a fixed number of operations:

- Read and validate one choice
- Check the required Energy or Focus
- Update the player statistics
- Calculate challenge damage
- Select an enemy response
- Reduce the challenge time
- Check the ending conditions

These operations take $O(1)$ time for one normal round.

Let a represent the number of menu choices made during an encounter. If the player only selects normal actions, the total encounter time is $O(a)$.

However, later encounters allow the player to open the inventory. One inventory use may require $O(n)$ time. In the worst case, if inventory work happens repeatedly, the encounter complexity can be written as:

$$O(a \times n)$$

The challenges currently have a fixed time of around six or seven turns. Therefore, encounters are small in the present game. The variable `a` is still used because invalid input, insufficient Energy, and inventory usage may repeat the menu without reducing the challenge time.

The local encounter variables, such as the selected option, turn Boolean, and temporary statistical structures do not grow with the number of rounds. The encounter therefore uses $O(1)$ auxiliary space, excluding the player's existing inventory.

2.5.6. Exploration Complexity

Exploration also uses a repeated menu. Each normal choice checks the current location, checks several Boolean flags, and updates the player's statistics. These are constant-time operations.

Let e represent the number of choices made before the player returns home. Without inventory or shop operations, the exploration takes $O(e)$ time.

The player may also open the inventory or purchase items. These operations can take $O(n)$ time. In the worst case, the complete exploration can therefore take $O(e \times n)$ time.

The player controls how long exploration continues. They may travel between APU and KLCC several times, enter shops repeatedly, or keep opening the inventory. For this reason, there is no fixed number of exploration iterations.

The exploration function only stores one location value, one menu option, statistic snapshots, and a fixed number of Boolean flags. Its auxiliary space complexity is $O(1)$.

2.5.7. Checkpoint and Retry Complexity

The `copyPlayerData()` function copies the normal Player values in constant time. However, the inventory must be copied node by node.

If the source inventory contains n nodes, the function visits every node and inserts a separate copy into the checkpoint inventory. The time complexity is therefore $O(n)$.

The checkpoint contains its own copy of all inventory nodes. Its space complexity is also $O(n)$.

Before copying, the destination inventory may also need to be cleared. Clearing and copying are both linear operations. Therefore

$$O(n) + O(n) = O(n)$$

During the Final Examination section, more than one checkpoint may exist. For example, there may be an active Player, a Week 13 checkpoint, and a Final Exam checkpoint. This may produce approximately $3n$ inventory nodes. Since constant values are removed in Big O notation:

$$O(3n) = O(n)$$

Let q represent the number of attempts made by the player. If one stage requires C time and restoring its checkpoint takes $O(n)$, the retry time can be represented as:

$$O(q \times (C + n))$$

The player decides how many times to retry, so there is no fixed worst-case number of attempts. However, retries do not create a new checkpoint every time. The same checkpoint is reused, so the space complexity remains $O(n)$ rather than $O(qn)$.

2.5.8. Complexity Summary

Operation	Best-Case Time	Worst-Case Time	Space Complexity
`isEmpty()`	$O(1)$	$O(1)$	$O(1)$
`insertAtEnd()`	$O(1)$	$O(1)$	$O(1)$ per new node
`addItem()`	$O(1)$	$O(n)$	$O(1)$ auxiliary
`getItemAt()`	$O(1)$	$O(n)$	$O(1)$ auxiliary
`displayInventory()`	$O(1)$	$O(n)$	$O(1)$ auxiliary
`removeItemAt()`	$O(1)$	$O(n)$	$O(1)$ auxiliary
`clear()`	$O(1)$	$O(n)$	$O(1)$ auxiliary
`useItem()`	$O(1)$	$O(n)$	$O(1)$ auxiliary
`openInventory()`	$O(1)$ when empty	$O(n)$	$O(1)$ auxiliary
Shop array access	$O(1)$	$O(1)$	$O(1)$
Purchase item	$O(1)$	$O(n)$	$O(1)$ auxiliary
One random event	$O(1)$	$O(1)$	$O(1)$
`copyPlayerData()`	$O(1)$ when empty	$O(n)$	$O(n)$
Encounter loop	$O(a)$	$O(a \times n)$	$O(1)$ auxiliary
Exploration loop	$O(e)$	$O(e \times n)$	$O(1)$ auxiliary
Retry system	One attempt	$O(q \times (C + n))$	$O(n)$

Table 1: Complexity Summary Table

Overall, most gameplay actions have constant-time complexity. The main operations that grow with the input size are the linked-list searches, traversals, and checkpoint copies. If A represents all menu choices made during the game and N represents the largest inventory size, a general worst-case time complexity is $O(A \times N)$.

The main variable-sized structure is the linked-list inventory. The Player and checkpoint objects may contain separate inventory copies, but only a fixed number of them exist at one time. Therefore, the overall space complexity is $O(N)$. In the current version, N is limited to six different item types. The practical cost is therefore small. The general analysis is

still useful because it shows how the program would behave if the inventory and item catalogue were expanded.

3. System Implementation

(Contributed by: AHMED UWAIS IBRAHIM RASHEED)

This section explains how the designs and algorithms discussed in Section 2 were converted into a working C++ program. The focus here is on the connection between files, functions, and gameplay systems. Selected code snippets are included where they help explain an important implementation decision.

3.1. File and class organization

Survive the Semester was developed as a multi-file C++ console application in Microsoft Visual Studio. As the game became larger, keeping all the code in `main.cpp` would have made it difficult to locate problems or add new events. Therefore, the program was divided according to the responsibility of each system.

File	Main responsibility
`main.cpp`	Controls the semester sequence, checkpoints, and retry decisions
`Player.h`	Stores player data and implements player-related actions
`Challenge.h`	Represents the state of an encounter
`Item.h`	Defines the common structure of an item
`Inventory.h`	Implements the single linked-list inventory
`shop.cpp/shop.h`	Implements the campus and downtown shops
`Encounters.cpp/Encounters.h`	Contains the turn-based challenges
`exploration.cpp/exploration.h`	Controls Week 2, Week 6, and Week 13 exploration
`Validations.cpp/Validations.h`	Repairs invalid input streams
`Display.cpp/Display.h`	Controls the slow text output and skipping behavior
`Ascii.cpp/Ascii/h`	Displays the title and location illustrations

Table 2: File Responsibility Table

`main.cpp` acts as the overall controller. It does not contain the internal details of every shop or encounter. Instead, it creates the Player, calls the required function for the current week, and processes the result returned by that function.

The Player object is passed to shops, encounters, and exploration functions using a pointer:

```
int playerWin = phoneDistractionChallenge(student);
int explorationResult = weekTwoExploration(student);
```

Passing the same Player pointer was important because every part of the game must modify the same player state. If a function received a separate Player copy, changes such as money earned, items purchased, or Energy lost would disappear when the function ended.

Most class functions are defined inside their header files. For a larger software project, separating every declaration and implementation would be cleaner. However, for this assignment, keeping the small Player, Challenge, and inventory operations close to their class definitions made the relationships easier to follow.

The display files were separated from the gameplay logic for a similar reason. `slowPrint()` controls how the narrative appears, while the encounter functions decide what happens to the player. This allowed us to change the presentation without rewriting the encounter calculations.

3.2. Inventory Implementation

The inventory is stored directly inside the Player class:

```
class Player
{
public:
    string name;
    float focus;
    float knowledge;
    float energy;
    float motivation;
    float luck;
    int money;
    InventoryLinkedList inventory;
    bool hasPartTimeJob;
    int studyNotesAvailable;
```

This means that each Player owns one inventory. Shop functions add items to it, while the Player class controls item usage. Keeping these responsibilities connected prevented unrelated parts of the program from directly changing item effects.

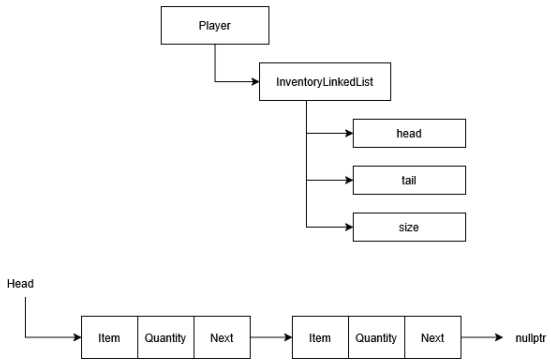

Figure 5: Implementing Inventory Linked-List inside Player

When an item is added, `addItem()` first searches the current list:

```
void addItem(Item item, int quantity = 1)
{
    if (quantity <= 0)
    {
        return;
    }
    InventoryNode* current = head;
    while (current != nullptr)
    {
        if (current->item.name == item.name)
        {
            current->quantity += quantity;
            cout << item.name << " was added to your inventory!" << endl;
            cout << "Current Quantity : " << current->quantity << endl;
            return;
        }
        current = current->next;
    }
    insertAtEnd(item, quantity);
    cout << endl;
    cout << item.name << " was added to your inventory!" << endl;
}
```

This implementation solved a problem that appeared when the player bought the same product more than once. Without the search, every Coffee purchase would create another Coffee entry. Instead, the existing node is found and its quantity is increased.

If no match is found, `insertAtEnd()` creates a new node. The empty-list case is handled separately because both `head` and `tail` must point to the first node:

```
void insertAtEnd(Item item, int quantity)
{
    InventoryNode* newNode = new InventoryNode;
    newNode->item = item;
    newNode->quantity = quantity;
    newNode->next = nullptr;

    if (head == nullptr)
    {
        head = tail = newNode;
    }
    else
    {
        tail->next = newNode;
        tail = newNode;
    }
    size++;
}
```

The Player interacts with the list through `openInventory()`. The displayed menu begins from 1, but `getItemAt()` uses a zero-based position. Therefore, the selected menu option is converted using `option - 1`.

After an item is selected, `useItem()` copies its effects into the player statistics and calls `checkStats()`. The quantity is then reduced:

```
focus += usedItem.focusRegen;
knowledge += usedItem.knowledgeRegen;
energy += usedItem.energyRegen;
motivation += usedItem.motivationRegen;
checkStats();

selected->quantity--;
cout << endl << "==== Item Result ====" << endl;
displayStatChanges(before);
cout << "========================" << endl;
if (selected->quantity <= 0)
{
    inventory.removeItemAt(index);
}
```

Removing the node at zero quantity keeps the inventory display clean. The player does not see entries such as `Coffee x0`, and the list only contains items that are still owned.

One implementation detail we had to be careful about was deleting the first or final node. Removing the first node changes `head`, while removing the final node also changes `tail`. When the only node is removed, both pointers become `nullptr`. Missing one of these updates could leave the inventory pointing to memory that had already been deleted.

The destructor calls `clear()` when the inventory is destroyed. `clear()` moves through the nodes and deletes each one. This is required because the nodes were created dynamically using `new`.

```
void clear()
{
    while (head != nullptr)
    {
        InventoryNode* current = head;
        head = head->next;
        delete current;
    }
    tail = nullptr;
    size = 0;
}
```

3.3. Shop and Item Implementation

Both shops use the same `Item` structure, but each one creates a different fixed array of products. For example, the downtown shop contains:

```
Item items[3] =
{
    { "Energy Drink", 8, -3, 0, 25, 0 },
    { "Study Notes", 12, 5, 3, 0, 0 },
    { "Instant Noodles", 4, 0, 0, 10, 0 }
};
```

The menu choice is linked to the array through `items[option - 1]`. This keeps the shop code short because one purchase function can process any item from either catalogue.

The purchasing logic is placed in `buyShop()`:

```
bool buyShop(Player* student, Item item)
{
    if (student->spendMoney(item.price))
    {
        student->inventory.addItem(item);
        cout << "Money Left : RM " << student->money << endl;
        return true;
    }
    return false;
}
```

`spendMoney()` performs the money check before the item is added. This ordering is important. The player should not receive an item if the payment fails.

`buyShop()` returns a Boolean because the downtown shop needs to know whether Study Notes were successfully purchased. Its availability is reduced only after a successful transaction:

```
bool purchaseSuccessful = buyShop(student, items[option - 1]);
if (purchaseSuccessful)
{
    --student->studyNotesAvailable;
}
```

Without this return value, the available quantity could decrease even when the player did not have enough money. The remaining stock is stored in Player because it must continue to exist after the player leaves the shop and visits it again during a later week.

The shop menu also calls `openInventory()`. We included this option so that a player with low Energy could buy and immediately use an item without leaving the shop and navigating through another menu.

3.4. Encounter Implementation

Each encounter creates a Challenge object containing its name, level, pressure, and remaining time. Although the individual choices are different, every challenge follows the same main condition:

```
while (
    phoneDistraction->challengeLevel > 0 &&
    student->focus > 0 &&
    phoneDistraction->challengeTime > 0)
{
```

The three conditions represent the main encounter endings. The player wins when the challenge level reaches zero, loses when Focus reaches zero, and receives a timeout result when the available challenge time ends.

At the beginning of each round, the current values are saved:

```
while (
    phoneDistraction->challengeLevel > 0 &&
    student->focus > 0 &&
    phoneDistraction->challengeTime > 0)
{
    int option1;
    bool isYourTurn = true;

    PlayerStats playerBefore = student->tempStat();
    ChallengeStats challengeBefore = phoneDistraction->tempStat();
```

These values are placed inside the loop because the output must compare the beginning and ending values of one round. When we placed the comparison outside the round logic, the output was less useful because it only showed the overall encounter change.

Player actions are processed using a `switch` statement. An action can reduce the challenge level, consume Energy, increase Knowledge, or recover a statistic. Some actions calculate their effect using several player statistics. This allows preparation during exploration to influence later encounters rather than making every action produce one fixed amount.

```
phoneDistraction->challengeLevel -= (((student->energy / 100.0) *
                                     (student->motivation * (student->focus) * 0.01)) +
                                     (student->knowledge / 10.0));
```

The `isYourTurn` Boolean controls whether the challenge is allowed to respond after the player's action. For example, continuing to study during the Phone Distraction encounter allows another distraction to occur. Turning off the phone damages the challenge without giving it another move.

After the player acts, the challenge is checked before its response is processed:

```
if (phoneDistraction->isDead())
{

    --phoneDistraction->challengeTime;
    slowPrint("You beat the Phone Distraction!");
    cout << endl;
    student->checkStats();
    cout << "==== Battle Result ====" << endl;
    student->displayPlayerStats();
    cout << endl;
    phoneDistraction->displayChallengeStats();
    delete phoneDistraction;
    return 1;
    break;
}
```

This check prevents a defeated challenge from damaging the player after its level has already reached zero.

If the challenge is still active, a random response is selected. After the response, the challenge time is reduced, player statistics are limited using `checkStats()`, and the saved values are compared with the new values.

Every encounter uses the same result codes:

Result	Meaning
1	The challenge was completed
2	Time ended, but the player still had Focus
3	The player lost all Focus

Table 3: Return Code Summary Table

These codes made the connection with `main.cpp` simpler. The main program does not need to inspect every Challenge object. It only checks the returned value and decides whether to continue, retry, or end the game.

Challenge objects are created using `new`, so every exit path deletes the object before returning. This includes victory, timeout, and player-defeat paths. We added these deletions carefully because encounter functions contain several possible return statements.

3.5. Exploration Implementation

Exploration required a different implementation from encounters because the player can move between locations and return to earlier menus. Each exploration function therefore uses a loop controlled by `currentLocation` and `explorationFinished`.

Week 2 begins with several local state variables:

```
int currentLocation = 0;
bool firstLocationChosen = false;
bool explorationFinished = false;
bool attendedClass = false;
bool missedClass = false;
bool exploredCampus = false;
bool exploredDowntown = false;
bool searchedPartTime = false;
```

`currentLocation` determines whether the APU or KLCC menu is displayed. Travelling changes this value, and the next loop iteration opens the menu for the new location.

The Boolean values remember what has happened during the current day. For example, `exploredCampus` prevents the player from repeating a random campus event to collect unlimited rewards. The `missedClass` value records whether the class is no longer available.

This was important when the player travelled from APU to KLCC before attending class. Simply returning to APU should not reset the time and reopen the class. Once `missedClass` becomes true, the program continues to block that option.

Some information must remain after the exploration function finishes. The part-time job is one example. It is stored as `student->hasPartTimeJob` rather than a local variable because Week 6 and Week 13 need to know whether the job was found earlier. Temporary events such as exploring the campus remain local because they only apply to the current exploration period.

Energy checks are performed before class, work, and exploration activities. When Energy reaches zero, these activities are blocked. However, the player can still open the inventory, visit a shop, or return home. We kept these recovery choices available because blocking the entire menu would leave the player trapped.

Random exploration events are processed after the action passes its Energy and repetition checks. The selected event changes the player statistics, and the program immediately calls `checkStats()` and displays the result. If Focus reaches zero, the exploration function returns `3`, allowing the checkpoint system in `main.cpp` to handle the defeat.

3.6. Checkpoint and Retry Implementation

Checkpoints are implemented using separate Player objects. Before an important stage begins, a new Player is created, and `copyPlayerData()` stores the current state inside it.

The normal Player values are copied individually. The destination inventory is then cleared and rebuilt:

```
void copyPlayerData(Player* originalPlayer, Player* copiedPlayer)
{
    copiedPlayer->name = originalPlayer->name;
    copiedPlayer->focus = originalPlayer->focus;
    copiedPlayer->knowledge = originalPlayer->knowledge;
    copiedPlayer->energy = originalPlayer->energy;
    copiedPlayer->motivation = originalPlayer->motivation;
    copiedPlayer->luck = originalPlayer->luck;
    copiedPlayer->money = originalPlayer->money;
    copiedPlayer->studyNotesAvailable = originalPlayer->studyNotesAvailable;
    copiedPlayer->hasPartTimeJob = originalPlayer->hasPartTimeJob;
    copiedPlayer->inventory.clear();


    InventoryNode* current = originalPlayer->inventory.head;

    while (current != nullptr)
    {
        copiedPlayer->inventory.insertAtEnd(current->item, current->quantity);

        current = current->next;
    }
}
```

We did not use direct assignment between the two Player objects because the inventory contains pointers. Directly copying those addresses would cause the checkpoint and active player to share the same inventory nodes. Using an item after the checkpoint was created could then change both objects.

Rebuilding the inventory creates independent nodes with identical contents. The checkpoint remains unchanged while the active player buys or consumes items.

Each stage is placed inside a Boolean-controlled loop. A simplified form is:

```
bool weekFinished = false;
while (!weekFinished)
{
    int result = encounter(student);
    switch (result)
    {
    case 1:
        weekFinished = true;
        break;
    case 2:
        weekFinished = true;
        break;
    case 3:
        // retry or quit
    }

}
```

We used loops instead of restarting `main()` or using `goto`. The loop keeps the retry behavior close to the stage it controls and makes it clear when the stage has finished.

The checkpoint copies not only the main statistics and inventory, but also progression values such as `hasPartTimeJob` and `studyNotesAvailable`. These values were easy to overlook because they are not part of the visible combat statistics. However, failing to restore them would produce an incorrect retry. A player could lose purchased Study Notes without restoring the shop stock, or lose a part-time job that had already been obtained before the checkpoint.

```
case 3:
    cout << endl;
    slowPrint("You could not survive Week 1.");

    if (askRetryCurrentWeek())
    {
        copyPlayerData(weekOneCheckpoint, student);

        cout << endl;
        slowPrint("Restarting Week 1...");
    }
    else
    {
        delete weekOneCheckpoint;
        delete student;

        return 0;
    }

    break;
```

The Final Examination has two checkpoint choices. `finalExamCheckpoint` repeats only the examination, while `weekThirteenCheckpoint` restores the player to the start of the final preparation period. The outer semester loop then repeats Week 13, Week 14, and the examination.

Once a checkpoint is no longer required, it is deleted. The active Player is also deleted before the program exits. Since the inventory destructor clears its nodes, deleting a Player also releases its separate linked-list memory.

3.7. Validation and Boundary Handling

Most game menus require a number. If the player enters text, `cin` enters a failure state and continues rejecting subsequent input unless the stream is repaired.

The shared validation function handles this case:

```
bool isInputValid()
{
    if (cin.fail())
    {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << endl << "Invalid Input!" << endl;
        return false;
    }
    return true;
}
```

`cin.clear()` removes the failure state, while `cin.ignore()` removes the unread invalid characters. The current menu then uses `continue` or returns control to the caller.

A separate range check is still required. Input such as `9` is a valid integer, so it does not cause `cin.fail()`, but it may not belong to a menu containing only options `1` to `3`. These cases are handled through conditions and the `default` branch of `switch` statements.

The name of the player also has a boundary check. `setName()` rejects an empty name or one containing only spaces. This prevents the Player from being created with a blank identity.

```
inline string setName()
{
    string name;
    bool isNameCorrect = false;
    while (!isNameCorrect)
    {
        name.clear();
        cout << "Enter your name:";
        getline(cin, name);

        isNameCorrect = name.find_first_not_of(" \t\r\n") != string::npos;
    }

    return name;
}
```

Other checks are kept close to the systems they protect:

- `spendMoney()` prevents purchases when money is insufficient.
- Exploration checks Energy before class, work, or exploration.
- `getItemAt()` rejects invalid linked-list positions.
- `openInventory()` checks for an empty inventory and invalid item choices.
- The downtown shop checks whether Study Notes are sold out.
- Boolean flags block repeated exploration rewards.
- `checkStats()` limits statistics after actions and item usage.
- Encounters check for victory before allowing another challenge response.

Keeping these checks near the affected code made debugging easier. For example, a money error could be checked inside Player and shop code without searching through every exploration function.

Overall, the completed implementation supports the intended semester progression from Week 1 to the Final Examination. The inventory, shops, encounters, and exploration sections share a single Player state, while checkpoints allow individual stages to be repeated without restarting the entire game.

4. Conclusion

(Contributed by: AHMED UWAIS IBRAHIM RASHEED)

When we first planned **Survive the Semester**, we wanted to create something more personal than a normal text-based RPG. Instead of fighting fictional monsters, the player faces problems that students can recognize, such as phone distraction, difficult lectures, surprise quizzes, group assignments, procrastination, and the pressure of a final examination. We aimed to maintain the basic turn-based RPG structure while giving it a setting that feels relatable to university students.

The completed project allows the player to experience selected stages of a semester through encounters, exploration, shops, items, and checkpoints. The systems are connected through one Player object, so decisions made earlier can affect later parts of the game. For example, attending class can improve Knowledge, working can provide money for items, and using those items can help the player survive a difficult encounter. We believe this connection between the different systems is one of the strongest parts of the project because the player's choices have consequences beyond the current menu.

The inventory was the main practical use of a singly linked list. Developing it helped us understand that linked lists are not only about connecting nodes. We also had to consider what happens when the first, last, or only node is removed, how duplicate items should be handled, and how dynamically allocated memory should be released. Using a quantity value inside each node prevented the inventory from creating several separate entries for the same item.

The checkpoint system was another important learning experience. Since the inventory contains pointers, copying a Player was not as simple as copying normal variables. The active player and checkpoint needed separate inventory nodes, which led to the implementation of a deep-copy function. This made concepts such as pointer ownership and shallow copying much clearer to us because an incorrect copy would directly affect item restoration and memory management.

The project also improved our understanding of program state. Some values, such as whether a location has already been explored, are only required during one exploration period. Other values, including the part-time job and remaining Study Notes, must continue across

several weeks. Deciding where these values should be stored taught us that variable scope can affect the complete gameplay flow, not only the function where the variable is created.

The final scope is smaller than our original idea. We initially wanted to include more weeks, locations, items, and story choices. Due to the available development time, we focused on selected weeks that represent the beginning, middle, and end of the semester. The current game therefore provides a complete journey, but some weeks are passed through short narrative transitions. The number of items and exploration events is also limited, so repeated playthroughs may eventually produce familiar situations.

There are also technical limitations. Several encounters follow similar structures but are written as separate functions, which creates repeated code. The exploration functions also became long because they manage locations, choices, conditions, and consequences in the same place. In addition, checkpoints only exist while the program is running, so the player cannot close the game and continue later. The checkpoint copy must also be updated manually whenever a new persistent Player variable is introduced.

In the future, we would add a permanent save-and-load system, more semester events, additional locations, and a wider variety of items. The encounter system could be redesigned around one reusable controller, while each challenge stores only its unique actions and values. Exploration events could also be stored separately from the main menu logic, making new content easier to add. A more flexible probability system would provide better control over random events and game balance.

Overall, **Survive the Semester** achieved the main goal we had at the beginning: creating a small but complete student-life RPG in which data structures and algorithms support actual gameplay. The project is not as large as we first imagined, but reducing the scope allowed the main systems to be completed and connected properly. More importantly, it helped us understand how pointers, dynamic memory, algorithms, and state management work together inside a complete program. For us, that understanding is the most valuable outcome of the project.

5. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). The MIT Press.

Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional. Retrieved from https://chenweixiang.github.io/docs/The_C++_Programming_Language_4th_Edition_Bjarne_Stroustrup.pdf

Tarek, A. (2006). A Generalized Set Theoretic Approach for Time and Space Complexity Analysis of Algorithms and Functions. *10th WSEAS International Conference on Applied Mathematics*, (pp. 316-324). Retrieved from https://www.academia.edu/6535395/A_Generalized_Set_Theoretic_Approach_for_Time_and_Space_Complexity_Analysis_of_Algorithms_and_Functions#abstract

Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.

6. Appendix

6.1. Reflective Essays

6.1.1. Member 1: MIN NYO SIN

Before this assignment, I already had some programming experience, but Data Structures and Algorithms felt like the point where programming became much deeper. Concepts such as linked lists, dynamic memory, and complexity analysis were confusing at first, and I often had to revisit the same topic before it finally made sense. There were times when I worried that I had made the project too ambitious. However, I was also excited because I finally had the opportunity to turn what I had learnt into something of my own.

Developing **Survive the Semester** was enjoyable, but it was rarely straightforward. The project started with a simple idea and gradually became much larger than I expected. Every new feature had to work with the systems that were already there, and sometimes fixing one problem created another somewhere else. Those moments were frustrating, especially when I could not immediately understand what had gone wrong. Like the student in the game, I sometimes had to manage my own motivation and pressure before I could continue. However, once I started working through a problem carefully, making progress became much easier.

The checkpoint system became the part of the project that I am most proud of. At first, I thought restoring the player would simply involve copying their information. I later discovered that the linked-list inventory made the process more complicated. It took several attempts before I understood why a normal copy did not work. When the checkpoint finally restored the correct statistics, money, and items after a failed week, I felt genuinely relieved. It was one of those “finally, it works” moments that made all the earlier frustration feel worthwhile.

I also enjoyed creating the statistic-change display. Although it was a smaller feature, I wanted players to clearly see the result of every decision. Finding a way to compare the statistics before and after an action gave me confidence because it was a solution I had worked out through my own attempts.

By the end of the project, I had learnt more than how to implement data structures. I learnt to be patient when a problem did not have an immediate answer, to reduce the scope when my ideas became too large, and to understand a problem before randomly changing the

code. Seeing the game progress from Week 1 to the Final Examination gave me a real sense of relief and achievement. The result is not perfect, but it is playable, represents my ideas, and shows how much I have improved. That is what makes this project personally meaningful to me.

6.1.2. Member 2: AHMED UWAIS IBRAHIM RASHEED

Procrastination was also an issue with mine, more specifically with starting. Continuing to work is very easy once I have started, but beginning to work is extremely difficult. Learning about and implementing a routine into my schedule worked wonders, and thanks to it, I barely "Survived the Semester". Having a set time to work helped me avoid leaving everything until the last minute and made the workload feel much more manageable. I also learned that I work much better when I only take a break closer to when I go to bed, which is very different from the "Split work into small sizes to do throughout the day" Advice that I usually hear.

I had finished my math game for the Mathematical Concepts for Games module relatively easily, and although I didn't go into this with the same expectations, I didn't think it would be that much harder. I soon came to realize that this was a team assignment, and trying to decipher someone else's code was a skill set I had to learn. When working on my own projects, I already know how my code works and why I wrote it in a certain way. However, when working in a team, I had to understand code that I had not written myself. This forced me to become better at reading and understanding existing code rather than only focusing on writing my own.

The project made me realize how important readable code is. When writing code for myself, I sometimes focused more on getting the program to work than on making the code easy to understand, like when I didn't name all the ASCII art functions by adding ascii to the end of them, which caused a few issues with the encounters script when it tried to call the procrastination encounter and also the ASCII art.

Overall, I feel that this project taught me lessons that I can use both within future projects and outside of programming. Within game development, I can utilize what I have learned about data structures, such as using linked lists for systems like an inventory, where items can be added and removed as needed. I also have a better understanding of working with C++, GitHub, and other people's code, which will be useful for future team projects. Outside

of the project, I want to continue using the routines that helped me manage my procrastination and stay consistent with my work. Although I still have many other areas that need improvement, many of which I don't even realize yet. I will use this experience as a stepping stone to overcome them.

6.2. Source Code and Flowchart Link

https://github.com/minnyosin/Data-Structures-and-Algorithms/tree/main/SurviveTheSemester

https://drive.google.com/file/d/1qq24a9j0MLtqaylBfWVHzAd0VBqEdmS6/view?usp=sharing

========== **END OF REPORT** ==========